Chroma

Querying Collections › Query and Get

Querying Collections

Query and Get

Copy page

Learn how to query and retrieve data from Chroma collections.

New Search API Available

Dense vector search, hybrid search, and more are available in the new powerful **Search API** for Chroma Cloud databases.

The Query API enables nearest-neighbor similarity search over dense embeddings. Use the Get API when you want to retrieve records without similarity ranking.

Python TypeScript Rust

Query

You can query a collection to run a similarity search using `.query`:

```
collection.query(
    query_texts=["thus spake zarathustra", "the oracle speaks"]
)
```

Chroma will use the collection's **embedding function** to embed your text queries, and use the output to run a vector similarity search against your collection.

Instead of providing `query_texts`, you can provide `query_embeddings` directly. You will be required to do so if your collection does not have an embedding function attached to it.

Chroma

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]]
)
```

By default, Chroma will return 10 results per input query. You can modify this number using the `n_results` argument:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100
)
```

The `ids` argument lets you constrain the search only to records with the IDs from the provided list:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100,
    ids=["id1", "id2"]
)
```

Both `query` and `get` support `where` for **metadata filtering** and `where_document` for **full-text search and regex**:

Chroma

```
    where_document={"$contains": "search string"} # query records with the search
)
```

Get

Use `.get` to retrieve records by ID and/or filters without similarity ranking:

```
collection.get(ids=["id1", "id2"]) # by IDs

collection.get(limit=100, offset=0) # with pagination
```

Results Shape

Chroma returns `.query` and `.get` results in **column-major** form (arrays per field). `.query` results are grouped per input query; `.get` results are a flat list of records.

```
    embeddings: Optional[List[List[Embeddings]]]
    documents: Optional[List[List[Document]]]
    uris: Optional[List[List[URI]]]
    metadatas: Optional[List[List[Metadata]]]
    distances: Optional[List[List[float]]]
    included: Include

class GetResult(TypedDict):
    ids: List[ID]
    embeddings: Optional[Embeddings]
    documents: Optional[List[Document]]
    uris: Optional[URIs]
    metadatas: Optional[List[Metadata]]
    included: Include
```

Here is a concrete example of what these responses look like in practice:

Chroma

```
  "documents": [["Chroma stores vectors.", "Embeddings power semantic search."]],
  "metadatas": [[
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ]],
  "distances": [[0.12, 0.21]],
  "included": ["embeddings", "documents", "metadatas", "distances"]
}
// Get result
{
  "ids": ["doc_1", "doc_7"],
  "embeddings": [[1, 2, 3, 4], [1, 2, 3, 4]],
  "documents": ["Chroma stores vectors.", "Embeddings power semantic search."],
  "metadatas": [
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ],
  "included": ["documents", "metadatas"]
}
```

In the results from the Get operation, corresponding elements in each array belong to the same document.

Python TypeScript Rust

```
result = collection.get(include=["documents", "metadatas"])
for id, document, metadata in zip(result["ids"], result["documents"], result["me
    print(id, document, metadata)
```

Query is a batch API and returns results grouped per input. A common pattern is to iterate over each query's "batch" of results, then iterate within that batch.

Chroma

```
for id, document, metadata in zip(ids, documents, metadatas):
    print(id, document, metadata)
```

Choosing Which Data is Returned

By default, Query returns `documents`, `metadatas`, and `distances`, and Get returns `documents` and `metadatas`.

Use `include` to control what comes back. `ids` are always returned.

Python TypeScript Rust

```
collection.query(
    query_texts=["my query"],
    include=["documents", "metadatas", "embeddings"],
)

collection.get(include=["documents"])
```

Was this page helpful? Yes No Suggest edits

Configure Collections
Previous

Metadata Filtering
Next

Chroma

Enterprise

Pricing

Changelog